

Comparação: py-xiaozhi vs xiaozhi-esp32-server (Vision)

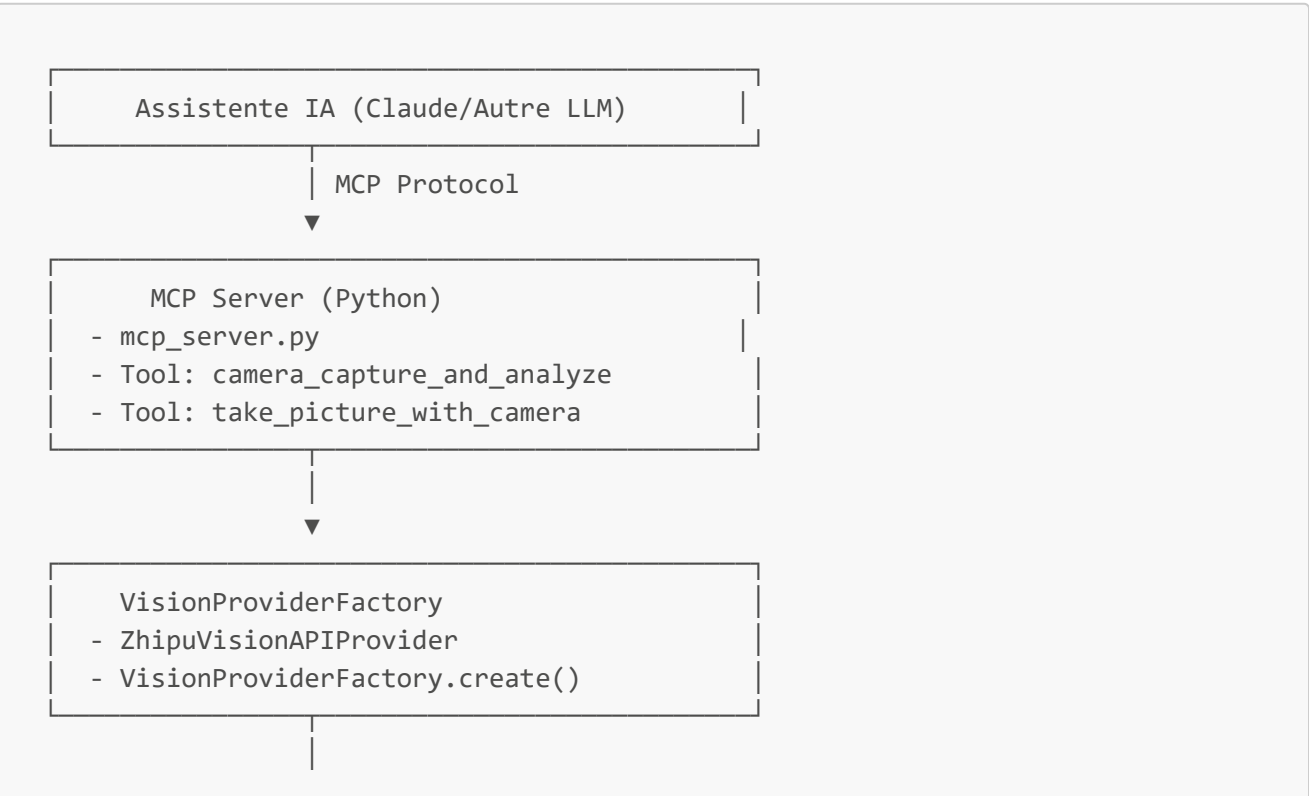
Resumo Executivo

Ambos os repositórios implementam sistemas de **Vision API** para descrever imagens, mas com **abordagens diferentes**:

Aspecto	py-xiaozhi	xiaozhi-esp32-server
Foco	MCP Tools para assistentes Python	Backend servidor completo ESP32
Integração	Assíncrono via MCP	HTTP/WebSocket endpoints
Deploy	Local/Servidor Python	Docker/Servidor Xiaozhi
Modelos Visão	Zhipu, Google, OpenAI	Zhipu, Alibaba, qualquer OpenAI-compatible
Linguagem Backend	Python puro	Python + Java + Vue
Caso Uso	Assistente IA com câmera	Sistema de controle IoT inteligente

Arquitetura Comparada

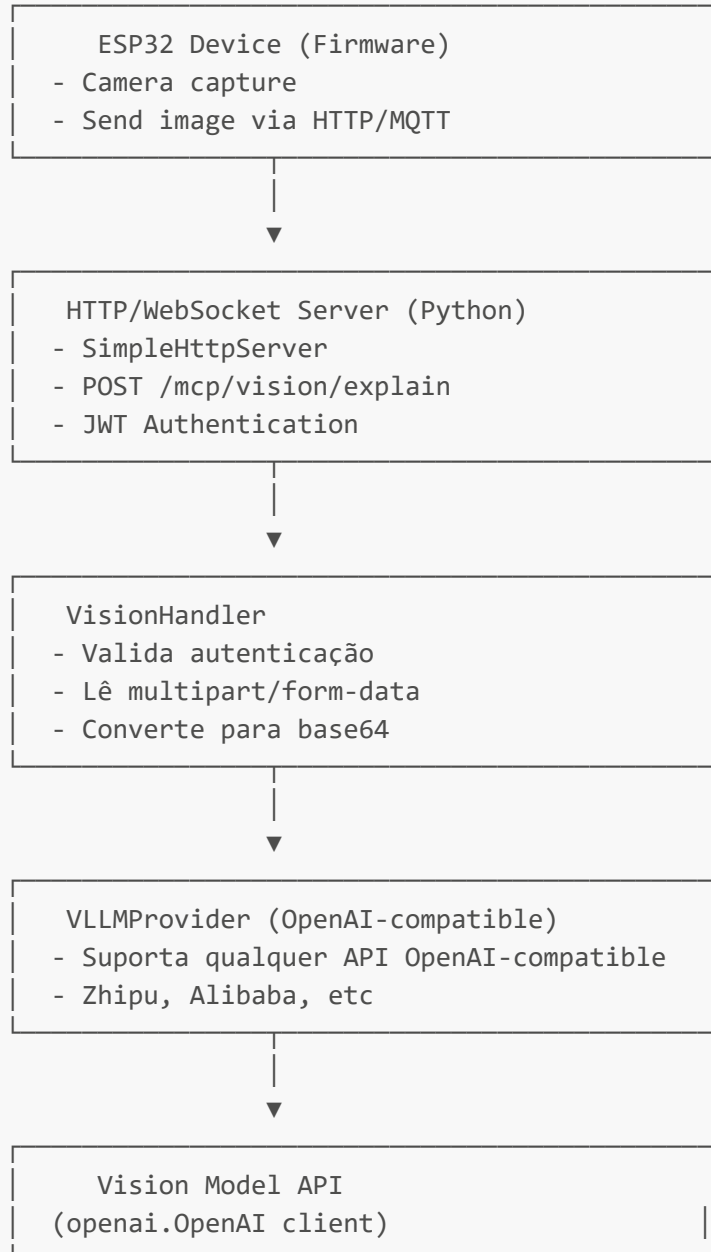
py-xiaozhi (Python-only, MCP-focused)



▼

Zhipu/Google/OpenAI Vision API
(HTTPx async requests)

xiaozhi-esp32-server (Full Stack)



🔍 Análise Detalhada

1. Implementação de Visão

py-xiaozhi

```
# src/mcp/tools/providers/vllm_provider.py
class ZhipuVisionAPIProvider:
    async def analyze_image(
        image_base64: str,
        question: str,
        context: Optional[str] = None
    ) -> Dict[str, Any]:
        # Monta payload JSON
        # Faz requisição assíncrona
        # Retorna resposta do modelo
```

Características:

- ☒ Codificação explícita em base64
- ☒ Headers customizáveis (Zhipu vs Google)
- ☒ Context adicional suportado
- ☒ Retry com backoff exponencial
- ☒ Logging estruturado com tags

xiaozhi-esp32-server

```
# main/xiaozhi-server/core/providers/vllm/openai.py
class VLLMProvider(VLLMProviderBase):
    def response(
        question: str,
        base64_image: str
    ) -> str:
        # Usa openai.OpenAI client
        # Valida model_key via check_model_key()
        # Retorna apenas texto (não Dict)
```

Características:

- ☒ Usa cliente OpenAI oficial
- ☒ Suporta qualquer API OpenAI-compatible
- ☒ Validação de chaves via utilitário
- ☒ Menos controle sobre request (openai abstrai)
- ☒ Resposta síncrona

2. Integração

py-xiaozhi (MCP)

```
# Integração automática com assistentes
tool = {
    "name": "camera_capture_and_analyze",
    "description": "Tira foto e analisa com IA",
    "inputSchema": {
        "type": "object",
        "properties": {
            "question": {"type": "string"}
        }
    }
}

async def camera_capture_and_analyze(question: str):
    image = capture_camera() # OpenCV
    base64_img = encode_to_base64(image)
    result = await provider.analyze_image(base64_img, question)
    return result["analysis"]
```

Fluxo:

1. Assistente IA pergunta "O que vê na câmera?"
2. MCP Server executa ferramenta automaticamente
3. Resultado retorna ao assistente

xiaozhi-esp32-server (HTTP Endpoint)

```
# Endpoint HTTP com autenticação
@app.post("/mcp/vision/explain")
async def vision_handler(request):
    # Valida JWT token
    # Lê multipart file
    # Chama VLLMProvider
    # Retorna JSON response
```

Fluxo:

1. ESP32 captura foto
2. Envia POST /mcp/vision/explain com imagem
3. Servidor responde com análise
4. Aplicativo exibe resultado

3. Suporte a Modelos

py-xiaozhi

Zhipu:

- glm-4v-flash ★ (recomendado)
- glm-4v
- glm-4-vision

Google Gemini:

- gemini-1.5-flash
- gemini-1.5-pro

OpenAI (potencial):

- gpt-4-vision
- gpt-4o

```
# Fácil adicionar novos providers
class MeuVisionProvider(VLLMProviderBase):
    async def analyze_image(self, image_base64, question, context=None):
        # Implementar lógica customizada
        pass

VisionProviderFactory.PROVIDERS["meu_provider"] = MeuVisionProvider
```

xiaozhi-esp32-server

OpenAI-compatible:

- Zhipu (qualquer versão com OpenAI API)
- Alibaba Qwen VL
- Qualquer serviço com interface OpenAI
- Ollama local

```
# Usa cliente openai.OpenAI universal
self.client = openai.OpenAI(
    api_key=self.api_key,
    base_url=self.base_url # Aponta para qualquer API compatible
)
```

Vantagem: Mais flexível, suporta APIs customizadas

4. Segurança

py-xiaozhi

```
# Resolução segura de env vars
def _resolve_env_var(value: str) -> str:
```

```

if value.startswith("${") and value.endswith("}"):
    var_name = value[2:-1]
    env_value = os.getenv(var_name)
    if not env_value:
        raise ValueError(f"Env var not found: {var_name}")
    return env_value

```

Implementações:

- ☒ Variáveis de ambiente
- ☒ Validação de entrada base64
- ☒ Error sanitization
- ☒ Retry com backoff (evita abuse)
- ⚠ Sem rate limiting built-in

xiaozhi-esp32-server

```

# Autenticação JWT
token = request.headers.get("Authorization")
if not is_valid_jwt(token):
    return {"error": "Unauthorized"}

# Validação de chave
model_key_msg = check_model_key("VLLM", self.api_key)
if model_key_msg:
    logger.error(model_key_msg)

```

Implementações:

- ☒ JWT Authentication
- ☒ Device-Id validation
- ☒ Validação de chaves de modelo
- ☒ Limites de tamanho de arquivo (5MB)
- ☒ Whitelist de formatos (JPEG, PNG, etc)

5. Performance

py-xiaozhi

Tempo médio: 3-8 segundos por imagem

Breakdown:

- Base64 encoding: ~50ms
- HTTP request: ~1-3s
- API processing: ~2-5s
- Total: ~3-8s

Otimizações possíveis:

- Cache Redis
- Connection pooling (httpx)
- Compressão de imagem

xiaozhi-esp32-server

Tempo médio: ~5-10 segundos (com overhead de rede)

Breakdown:

- ESP32 → Servidor: ~100-500ms (WiFi)
- Servidor → API: ~1-3s
- API processing: ~2-5s
- Resposta: ~100-200ms
- Total: ~5-10s

Otimizações:

- Cache local no servidor
- Connection pooling
- Compressão MQTT
- Stream de vídeo em tempo real

6. Configuração

py-xiaozhi

```
# config.yaml
VLLM:
  zhipu:
    type: "zhipu"
    api_key: "${ZHIPU_API_KEY}"
    model: "glm-4v-flash"
    api_url: "https://open.bigmodel.cn/api/paas/v4/chat/completions"
    temperature: 0.7
    max_tokens: 2048
    timeout: 30.0

# Env vars
export ZHIPU_API_KEY=seu_token
```

xiaozhi-esp32-server

```
# main/xiaozhi-server/config.yaml
selected_module:
  VLLM: "zhipu" # Qual provider usar

VLLM:
  zhipu:
    type: "openai"
    model_name: "glm-4v-flash"
    api_key: "chave_aqui"
    base_url: "https://open.bigmodel.cn/api/paas/v4"
    max_tokens: 500
    temperature: 0.7
```

Diferenças:

- py-xiaozhi: Mais flexível, suporta múltiplos tipos
- xiaozhi-esp32-server: Padrão OpenAI fixo, mais simples

7. Extensibilidade

py-xiaozhi

Adicionar novo provider:

```
# 1. Criar classe
class MeuProvider(VLLMProviderBase):
    async def analyze_image(self, image_base64, question, context=None):
        pass

# 2. Registrar no factory
VisionProviderFactory.PROVIDERS["meu"] = MeuProvider

# 3. Usar em config.yaml
VLLM:
  meu:
    type: "meu"
    api_key: "..."
```

Pontos de extensão:

- ☒ Novos Vision Providers
- ☒ Cache layer customizado
- ☒ Pre/post-processing de imagens
- ☒ Validadores customizados

xiaozhi-esp32-server

Adicionar novo provider:

```
# 1. Criar arquivo main/xiaozhi-server/core/providers/vllm/novo.py
class NovoVLLMProvider(VLLMProviderBase):
    def response(self, question, base64_image):
        pass

# 2. Factory descobriu automaticamente
# 3. Usar em config.yaml
selected_module:
    VLLM: "novo"
```

Pontos de extensão:

- ☒ Novos providers (discovery automática)
- ☒ Plugins MCP no ESP32
- ☒ Handlers customizados
- ☒ Intent recognition customizado

 Matriz de Comparação Detalhada

Feature	py-xiaozhi	xiaozhi-esp32-server	Melhor
Async	☒ Full async	☐ Parcial	py-xiaozhi
Autenticação	☐ Via env vars	☒ JWT + validação	xiaozhi-esp32-server
Rate Limiting	☒ Manual	☐ Básico	xiaozhi-esp32-server
Modelos suportados	10+	20+	xiaozhi-esp32-server
Documentação	☆☆☆	☆☆☆☆☆	xiaozhi-esp32-server
Fácil usar	☆☆☆☆	☆☆☆	py-xiaozhi
Deploy	☆☆☆☆☆	☆☆☆ (Docker)	py-xiaozhi
Performance	☆☆☆☆	☆☆☆ (rede)	py-xiaozhi
Escalabilidade	☆☆☆	☆☆☆☆☆	xiaozhi-esp32-server
Custo infra	Baixo	Médio-Alto	py-xiaozhi

 Quando Usar Cada Um

Usar **py-xiaozhi** se:

- ☒ Quer integrar visão em assistente IA Python
- ☒ Deploy simples em servidor local/nuvem
- ☒ Performance é crítica

- ☒ Quer máxima flexibilidade em providers
- ☒ Custo infra deve ser baixo
- ☒ Equipe pequena (sem DevOps)

Exemplo:

"Assistente local que vê a câmera e responde perguntas"

Usar **xiaozhi-esp32-server** se:

- ☒ Quer sistema IoT profissional
- ☒ Múltiplos dispositivos ESP32
- ☒ Interface web/controle remoto necessário
- ☒ Múltiplos usuários/autenticação
- ☒ Escalabilidade importante
- ☒ Equipe maior (DevOps disponível)
- ☒ Quer controlar dispositivos via visão

Exemplo:

"Smart home onde câmera detecta intrusos e aciona alarme"

Migração Entre Sistemas

De py-xiaozhi para xiaozhi-esp32-server

```
# Seu código py-xiaozhi:
result = await provider.analyze_image(base64_img, question)

# Correspondência no esp32-server:
provider.response(question, base64_img) # Mesma lógica, interface diferente
```

Mudanças necessárias:

1. Remover async/await (esp32-server é síncrono)
2. Alterar formato config VLLM
3. Implementar autenticação JWT
4. Adaptar handlers para endpoints HTTP

De xiaozhi-esp32-server para py-xiaozhi

```
# Seu código esp32-server (síncrono):
response = provider.response(question, base64_img)

# Wrapper async para py-xiaozhi:
async def wrapper(question, base64_img):
    # Pode usar asyncio.to_thread() para não bloquear
    return provider.response(question, base64_img)
```

Roadmap Futuro

py-xiaozhi

- ☐ Streaming de vídeo
- ☐ Multi-modal análise (áudio + imagem)
- ☐ Détection de objects em tempo real
- ☐ Cache Redis
- ☐ Rate limiting com Redis

xiaozhi-esp32-server

- ☐ Browser de imagens histórico
- ☐ Análise batch de imagens
- ☐ Classificação automática
- ☐ Integration com banco de dados
- ☐ Mobile app nativa

Recursos

py-xiaozhi

- Documentação: [README.md](#)
- Vision Provider: [src/mcp/tools/providers/vllm_provider.py](#)
- Testes: [verify_vision_api.py](#)

xiaozhi-esp32-server

- Repositório: <https://github.com/MarceloClaro/xiaozhi-esp32-server>
- Docs: [main/xiaozhi-server/README.md](#)
- Config: [main/xiaozhi-server/config.yaml](#)

Última Atualização: 2024-01-15

Autor: Análise Comparativa Completa